

بله. اگر قرار باشد SQL را در حد یک کلاس فشرده‌ی ۶۰ تا ۷۵ دقیقه‌ای ولی واقعاً سطح بالا یاد بگیری، باید از «SQL چیست؟» شروع کنیم و تا **JOIN, GROUP BY, Subquery, CTE, Window Function, Transaction, Index, Normalization** و **Performance** جلو برویم.

نکته مهم این است که SQL را نباید مثل مجموعه‌ای از دستورها حفظ کنی. باید بفهمی دیتابیس چطور فکر می‌کند. اگر این منطق را بگیری، بخش بزرگی از SQL برایت طبیعی می‌شود.

آموزش جامع SQL از صفر تا سطح خوب مصاحبه

نقشه‌ی کلی

تقریباً این مسیر را طی می‌کنیم:

1. Database چیست؟

2. SQL چیست؟

3. جدول، Row، Column، Key

4. ساخت Database و Table

5. SELECT

6. WHERE

7. فیلترها و Operatorها

8. ORDER BY

9. DISTINCT

10. توابع Aggregate

11. GROUP BY

12. HAVING

13. JOIN

14. INSERT / UPDATE / DELETE

15. NULL

16. CASE

17. Subquery

18. CTE

19. Set Operations

20. Window Functions

21. Primary Key / Foreign Key / Constraints

22. ACID و Transaction

Index .23

View .24

Normalization .25

Performance و Query Execution .26

SQL Injection .27

MySQL/PostgreSQL/SQL Server و تفاوت SQL .28

نکات مصاحبه‌ای .29

یک مثال جامع واقعی .30

1. اصلاً Database چیست؟

فرض کن یک فروشگاه اینترنتی داریم.

اطلاعات زیر را داریم:

- مشتری‌ها
- محصولات
- سفارش‌ها
- پرداخت‌ها
- موجودی
- کارکنان

می‌توانیم همه را داخل Excel نگه داریم.

اما وقتی داده زیاد شود، Excel مشکلات جدی پیدا می‌کند:

- داده تکراری می‌شود.
- ارتباط بین داده‌ها سخت می‌شود.
- چند کاربر همزمان مشکل ایجاد می‌کنند.
- امنیت ضعیف‌تر است.
- Query‌های پیچیده سخت می‌شوند.
- مدیریت میلیون‌ها رکورد مناسب نیست.
- Integrity داده دشوار می‌شود.

اینجاست که **Database Management System — DBMS** وارد می‌شود.

مثلاً:

- PostgreSQL
- MySQL
- Microsoft SQL Server
- Oracle Database
- SQLite

2. SQL چیست؟

SQL مخفف:

Structured Query Language

است.

یعنی:

زبان استاندارد برای تعریف، خواندن، تغییر و مدیریت داده‌های موجود در پایگاه‌های داده رابطه‌ای.

با SQL می‌توانی بگویی:

تمام مشتریان آلمانی که در سال 2026 بیش از 1000 یورو خرید داشته‌اند را نشان بده.

یا:

میانگین فروش هر ماه را محاسبه کن.

یا:

پرفروش‌ترین محصول هر کشور را پیدا کن.

یا:

کاربر جدیدی وارد دیتابیس کن.

یا:

قیمت محصول شماره 17 را تغییر بده.

SQL یک زبان **Declarative** است.

این نکته بسیار مهم است.

در Python معمولاً می‌گویی:

چگونه این کار انجام شود.

ولی در SQL بیشتر می‌گویی:

چه چیزی می‌خواهیم.

مثلاً:

```
SELECT name
FROM customers
WHERE country = 'Germany';
```

تو الگوریتم جست‌وجو را مشخص نکرده‌ای.

DBMS خودش تصمیم می‌گیرد چطور داده را پیدا کند.

3. SQL با Database چه فرقی دارد؟

این اشتباه بسیار رایج است:

MySQL همان SQL نیست. ❌

SQL یک زبان است.

ولی MySQL یک **Database Management System** است که SQL را پیاده‌سازی می‌کند.

مشابه این:

```
SQL = Language
PostgreSQL
MySQL
SQL Server
Oracle
SQLite
↓
Database Systems using SQL
```

البته هرکدام Dialect خاص خودشان را دارند.

مثلاً PostgreSQL و MySQL بعضی syntaxهای متفاوت دارند.

اما حدود ۸۰-۹۰٪ مفاهیم اصلی مشترک‌اند.

4. Relational Database یعنی چه؟

در Relational Database داده معمولاً در قالب **Table** ذخیره می‌شود.

مثلاً:

customers

customer_id	name	country	age
-------------	------	---------	-----

1	Ali	Iran	30
2	Anna	Germany	28
3	Sara	France	35

هر سطر:

Row / Record

هر ستون:

Column / Attribute

مثلاً:

```
Row:
2 | Anna | Germany | 28

Column:
country
```

5. Schema چیست؟

Schema ساختار منطقی دیتابیس است.

یعنی مشخص می‌کند:

- چه جدول‌هایی داریم؟
- هر جدول چه ستون‌هایی دارد؟
- نوع داده ستون چیست؟
- Primary Key چیست؟
- Foreign Key چیست؟
- چه Constraint‌هایی وجود دارند؟

مثلاً:

```
customers
-----
customer_id
name
email
country

orders
-----
order_id
customer_id
```

```
order_date
amount
```

6. Data Type چیست؟

هر ستون باید نوع داده داشته باشد.

مثلاً:

```
INT
VARCHAR
TEXT
DATE
TIMESTAMP
BOOLEAN
DECIMAL
FLOAT
```

مثال:

```
age INT
```

یعنی عدد صحیح.

```
name VARCHAR(100)
```

یعنی رشته تا تقریباً 100 کاراکتر.

```
price DECIMAL(10,2)
```

برای مقدارهایی مثل:

```
1250.75
```

مخصوصاً برای پول معمولاً DECIMAL از FLOAT مناسبتر است، چون Floating Point ممکن است خطای تقریب داشته باشد.

7. ساخت جدول

مثلاً:

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(200),
  country VARCHAR(100),
  age INT
);
```

داریم می‌گوییم جدولی به نام:

```
customers
```

بساز.

8. Primary Key چیست؟

Primary Key ستونی است که هر Row را به صورت یکتا شناسایی می‌کند.

مثلاً:

customer_id

جدول:

customer_id	name
1	Ali
2	Sara
3	Reza

این قابل قبول است.

ولی:

customer_id	name
1	Ali
1	Sara

نباید اتفاق بیفتد.

Primary Key

- Unique است.
- NULL نیست.
- هر رکورد را uniquely identify می‌کند.

9. اولین دستور اصلی: SELECT

مهم‌ترین دستور SQL:

SELECT

برای خواندن داده.

مثلاً:

```
SELECT name
FROM customers;
```

یعنی:

ستون name را از جدول customers نشان بده.

چند ستون

```
SELECT name, country, age
FROM customers;
```

همه ستونها

```
SELECT *
FROM customers;
```

* یعنی:

همه ستونها.

برای exploration خوب است.

ولی در production بهتر است بی دلیل از **SELECT *** استفاده نکنی.

مثلاً بهتر است:

```
SELECT customer_id, name, email
FROM customers;
```

چون:

- داده اضافی منتقل نمی شود.
- Query خواناتر است.
- تغییر Schema کمتر Query را خراب می کند.
- معمولاً بهینه تر است.

10. SQL Query را چگونه بخوانیم؟

مثلاً:

```
SELECT name, age
FROM customers;
```

خوانش:

از جدول customers، ستون‌های name و age را انتخاب کن.

تقریباً ساختار پایه:

```
SELECT columns
FROM table;
```

11. WHERE — فیلتر کردن

فرض کنیم فقط مشتریان آلمان را می‌خواهیم.

```
SELECT *
FROM customers
WHERE country = 'Germany';
```

WHERE یعنی شرط.

Operatorهای مهم

```
=
>
<
>=
<=
<>
!=
```

مثلاً:

```
SELECT *
FROM customers
WHERE age > 30;
```

12. AND / OR / NOT

چند شرط:

```
SELECT *
FROM customers
WHERE country = 'Germany'
AND age > 30;
```

یعنی:

```
German AND older than 30
```

OR

```
SELECT *
FROM customers
```

```
WHERE country = 'Germany'  
OR country = 'France';
```

NOT

```
SELECT *  
FROM customers  
WHERE NOT country = 'Germany';
```

یا:

```
WHERE country <> 'Germany'
```

13. مراقب AND و OR باش

مثلاً:

```
WHERE country = 'Germany'  
OR country = 'France'  
AND age > 30
```

به دلیل precedence معمولاً AND زودتر ارزیابی می‌شود.

بهنتر:

```
WHERE  
  (country = 'Germany' OR country = 'France')  
  AND age > 30;
```

استفاده از Parentheses بسیار مهم است.

BETWEEN .14

مثلاً افراد 20 تا 30 سال:

```
SELECT *  
FROM customers  
WHERE age BETWEEN 20 AND 30;
```

معمولاً Inclusive است:

```
20 <= age <= 30
```

IN .15

به جای:

```
WHERE country = 'Germany'
OR country = 'France'
OR country = 'Italy'
```

بنویس:

```
WHERE country IN ('Germany', 'France', 'Italy');
```

خیلی تمیزتر است.

16. LIKE — جست و جوی Pattern

مثلاً اسم با A شروع شود:

```
SELECT *
FROM customers
WHERE name LIKE 'A%';
```

% یعنی هر تعداد Character.

مثلاً:

```
Anna
Ali
Alexander
```

اگر اسم با son تمام شود:

```
WHERE name LIKE '%son'
```

اگر وسطش an داشته باشد:

```
WHERE name LIKE '%an%'
```

در SQL معمولاً _ یعنی دقیقاً یک کاراکتر:

```
WHERE name LIKE 'A_i'
```

ممکن است:

```
Ali
Ari
```

match شود.

17. DISTINCT

فرض کن country داریم:

```
Germany
Germany
France
France
Iran
Germany
```

اگر بزنیم:

```
SELECT country
FROM customers;
```

تکراری‌ها هم می‌آیند.

اما:

```
SELECT DISTINCT country
FROM customers;
```

خروجی:

```
Germany
France
Iran
```

ORDER BY .18

مرتب‌سازی:

```
SELECT *
FROM customers
ORDER BY age;
```

پیش‌فرض معمولاً Ascending است.

یعنی:

```
20
25
31
40
```

نزولی

```
ORDER BY age DESC;
```

صعودی

```
ORDER BY age ASC;
```

چند سطح

```
ORDER BY country ASC, age DESC;
```

ابتدا کشور.

داخل هر کشور، سن نزولی.

LIMIT .19

مثلاً فقط 10 نتیجه:

در PostgreSQL/MySQL:

```
SELECT *  
FROM customers  
LIMIT 10;
```

مثلاً:

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 5;
```

یعنی:

پنج محصول گران‌تر.

در SQL Server معمولاً:

```
SELECT TOP 5 *  
FROM products  
ORDER BY price DESC;
```

AS با Alias .20

مثلاً:

```
SELECT name AS customer_name  
FROM customers;
```

یا:

```
SELECT price * quantity AS total_price  
FROM order_items;
```

Alias برای خوانایی خروجی بسیار مفید است.

21. محاسبات داخل SELECT

SQL صرفاً داده را نمایش نمی‌دهد؛ محاسبات هم انجام می‌دهد.

```
SELECT
    product_name,
    price,
    price * 1.1 AS price_with_tax
FROM products;
```

22. Aggregate Functions

بسیار مهم.

توابع معروف:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

COUNT

تعداد مشتری:

```
SELECT COUNT(*)
FROM customers;
```

مثلاً:

```
1250
```

AVG

```
SELECT AVG(age)
FROM customers;
```

SUM

```
SELECT SUM(amount)
FROM orders;
```

کل فروش.

MAX

```
SELECT MAX(amount)
FROM orders;
```

بزرگترین سفارش.

MIN

```
SELECT MIN(amount)
FROM orders;
```

GROUP BY .23

اینجا SQL جدی تر می شود.

فرض کن:

orders

order_id	customer_id	country	amount
1	10	Germany	100
2	11	Germany	200
3	12	France	150
4	13	France	50

می خواهیم فروش هر کشور را بدانیم.

```
SELECT
  country,
  SUM(amount)
FROM orders
GROUP BY country;
```

خروجی:

country	sum
Germany	300
France	200

معنی GROUP BY:

رکوردهایی که مقدار مشخصی مشترک دارند را گروه‌بندی کن.

24. مثال مهم GROUP BY

تعداد مشتری در هر کشور:

```
SELECT
    country,
    COUNT(*) AS customer_count
FROM customers
GROUP BY country;
```

25. قانون مهم GROUP BY

این Query معمولاً مشکل دارد:

```
SELECT country, name, COUNT(*)
FROM customers
GROUP BY country;
```

چرا؟

چون برای Germany چند name وجود دارد.

DBMS نمی‌داند کدام name را برگرداند.

قاعده‌ی مفهومی:

چیزی که در SELECT آورده‌ای و Aggregate نیست، معمولاً باید در GROUP BY باشد.

مثلاً:

```
SELECT country, COUNT(*)
FROM customers
GROUP BY country;
```

درست است.

26. HAVING

فرق WHERE و HAVING از سؤال‌های کلاسیک مصاحبه است.

WHERE:

Rowها را قبل از Grouping فیلتر می‌کند.

HAVING:

Group ها را بعد از Grouping فیلتر می‌کند.

مثلاً کشورهایی که بیش از 100 مشتری دارند:

```
SELECT
    country,
    COUNT(*) AS customer_count
FROM customers
GROUP BY country
HAVING COUNT(*) > 100;
```

نمی‌نویسیم:

```
WHERE COUNT(*) > 100
```

چون Aggregate هنوز در مرحله WHERE ساخته نشده است.

27. ترتیب منطقی اجرای SQL

این قسمت بسیار مهم است.

وقتی می‌نویسی:

```
SELECT country, COUNT(*)
FROM customers
WHERE age > 18
GROUP BY country
HAVING COUNT(*) > 10
ORDER BY COUNT(*) DESC;
```

SQL از نظر منطقی تقریباً به این ترتیب پردازش می‌شود:

```
FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT
```

نه به ترتیبی که نوشته‌ای.

این یک نکته کلیدی برای فهم SQL است.

28. مهم‌ترین بخش JOIN: SQL

اگر JOIN را نفهمی، SQL را واقعاً بلد نیستی.

Relational Database بر اساس ارتباط جدول‌ها ساخته شده.

فرض کنیم:

customers

customer_id	name
1	Ali
2	Sara
3	Anna

:9

orders

order_id	customer_id	amount
101	1	200
102	1	300
103	3	150

customer_id ارتباط بین دو جدول است.

INNER JOIN .29

می‌خواهیم:

```
Order
Customer Name
Amount
```

بنویسیم:

```
SELECT
    orders.order_id,
    customers.name,
    orders.amount
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id;
```

خروجی:

order_id	name	amount
101	Ali	200
102	Ali	300
103	Anna	150

Table Alias .30

نسخه تمیزتر:

```
SELECT
    o.order_id,
    c.name,
    o.amount
FROM orders AS o
JOIN customers AS c
    ON o.customer_id = c.customer_id;
```

خیلی رایج است.

حتی معمولاً AS را برای Table Alias حذف می‌کنند:

```
FROM orders o
JOIN customers c
```

31. INNER JOIN دقیقاً چه می‌کند؟

فقط رکوردهایی را نگه می‌دارد که در هر دو سمت Match دارند.

مثلاً Sara هیچ سفارشی ندارد.

بنابراین در INNER JOIN دیده نمی‌شود.

32. LEFT JOIN

اگر بخواهیم همه مشتریان را ببینیم حتی اگر سفارش ندارند:

```
SELECT
    c.name,
    o.order_id
FROM customers c
```

```
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

خروجی تقریباً:

```
Ali    101
Ali    102
Sara    NULL
Anna    103
```

یعنی جدول سمت چپ کاملاً حفظ می‌شود.

INNER JOIN vs LEFT JOIN .33

این را باید کاملاً مسلط باشی:

```
INNER JOIN
ها Match فقط

LEFT JOIN
Left + Match ها همه
```

RIGHT JOIN .34

برعکس LEFT JOIN:

```
RIGHT JOIN
```

همه رکوردهای جدول سمت راست حفظ می‌شوند.

اما در عمل خیلی‌ها ترجیح می‌دهند Query را بازنویسی کنند و LEFT JOIN استفاده کنند چون خواناتر است.

FULL OUTER JOIN .35

تمام داده‌های دو جدول:

- Match ها
- فقط Left
- فقط Right

مثلاً PostgreSQL:

```
SELECT *
FROM a
FULL OUTER JOIN b
ON a.id = b.id;
```

MySQL به صورت مستقیم FULL OUTER JOIN ندارد.

CROSS JOIN .36

تمام ترکیب‌های ممکن.

اگر A سه Row و B چهار Row داشته باشد:

```
3 × 4 = 12 rows
SELECT *
FROM colors
CROSS JOIN sizes;
```

مثلاً:

```
Red S
Red M
Red L
Blue S
Blue M
Blue L
...
```

گاهی مفید است، ولی یک Cross Join اشتباه می‌تواند خروجی عظیمی تولید کند.

Self Join .37

یک جدول را با خودش Join می‌کنیم.

مثلاً:

employees

employee_id	name	manager_id
1	John	NULL
2	Sara	1
3	Ali	1

برای پیدا کردن مدیر هر Employee:

```
SELECT
    e.name AS employee,
    m.name AS manager
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.employee_id;
```

Foreign Key .38

در orders:

```
customer_id
```

به customers.customer_id اشاره می‌کند.

می‌توانیم بگوییم:

```
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  amount DECIMAL(10,2),

  FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

این باعث **Referential Integrity** می‌شود.

یعنی نمی‌توانی مثلاً سفارش برای:

```
customer_id = 999999
```

بسازی اگر چنین Customer ای وجود ندارد، البته بسته به تنظیمات Constraint.

INSERT .39

اضافه کردن Row:

```
INSERT INTO customers
  (customer_id, name, country, age)
VALUES
  (1, 'Ali', 'Iran', 30);
```

چند Row

```
INSERT INTO customers
  (customer_id, name, country, age)
VALUES
  (1, 'Ali', 'Iran', 30),
  (2, 'Sara', 'Germany', 28),
  (3, 'Anna', 'France', 35);
```

UPDATE .40

مثلاً:

```
UPDATE customers  
SET country = 'Germany'  
WHERE customer_id = 1;
```

نکته حیاتی:

اگر WHERE را حذف کنی:

```
UPDATE customers  
SET country = 'Germany';
```

ممکن است کشور تمام مشتری‌ها را Germany کند.

این یکی از اشتباهات خطرناک SQL است.

DELETE .41

```
DELETE FROM customers  
WHERE customer_id = 10;
```

بدون WHERE:

```
DELETE FROM customers;
```

تمام Rowهای جدول را پاک می‌کند.

خود جدول باقی می‌ماند.

DELETE vs DROP vs TRUNCATE .42

مصاحبه‌ای.

DELETE

Rowها را حذف می‌کند:

```
DELETE FROM customers  
WHERE country = 'Iran';
```

می‌تواند WHERE داشته باشد.

TRUNCATE

تقریباً تمام داده‌های جدول را سریع حذف می‌کند:

```
TRUNCATE TABLE customers;
```

رفتار دقیق Transaction/Identity به DBMS بستگی دارد.

DROP

خود Object را حذف می‌کند:

```
DROP TABLE customers;
```

یعنی:

```
Table + structure + data
```

از بین می‌رود.

پس:

DELETE	→ Rows
TRUNCATE	→ All Rows
DROP	→ Table itself

با جزئیات دقیق‌تر وابسته به DBMS.

ALTER TABLE .43

تغییر Structure:

```
ALTER TABLE customers
ADD phone VARCHAR(30);
```

یا مثلاً حذف Column:

```
ALTER TABLE customers
DROP COLUMN phone;
```

Syntax دقیق ممکن است بین DBMS ها متفاوت باشد.

Constraints .44

Constraint یعنی قانونی که روی داده اعمال می‌کنیم.

مهم‌ترین‌ها:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
DEFAULT
```

NOT NULL

```
name VARCHAR(100) NOT NULL
```

یعنی Name نمی‌تواند Null باشد.

UNIQUE

```
email VARCHAR(200) UNIQUE
```

ایمیل تکراری ممنوع.

CHECK

```
age INT CHECK (age >= 0)
```

DEFAULT

```
country VARCHAR(100) DEFAULT 'Unknown'
```

NULL. 45

یکی از بخش‌هایی که مبتدی‌ها شدیداً اشتباه می‌کنند.

NULL یعنی:

مقدار Unknown / Missing / Not Available

NULL برابر صفر نیست.

NULL برابر Empty String هم نیست.

مثلاً:

```
age = NULL
```

یعنی سن مشخص نیست.

46. چرا این اشتباه است؟

```
WHERE age = NULL
```

معمولاً درست نیست.

باید بنویسی:

```
WHERE age IS NULL;
```

یا:

```
WHERE age IS NOT NULL;
```

Three-valued Logic .47

SQL فقط:

```
TRUE  
FALSE
```

ندارد.

یک حالت سوم هم دارد:

```
UNKNOWN
```

مثلاً:

```
NULL = 5
```

نتیجه False نیست.

بلکه:

```
UNKNOWN
```

است.

همین باعث رفتارهای عجیب NULL می‌شود.

COALESCE .48

برای جایگزینی NULL.

مثلاً:

```
SELECT  
    name,  
    COALESCE(phone, 'No Phone')  
FROM customers;
```

اگر phone برابر NULL باشد:

```
No Phone
```

نمایش داده می‌شود.

COALESCE اولین مقدار non-null را برمی‌گرداند:

```
COALESCE(a, b, c, 0)
```

CASE .49

مثل if/else در SQL.

مثلاً:

```
SELECT
  name,
  age,
  CASE
    WHEN age < 18 THEN 'Minor'
    WHEN age < 60 THEN 'Adult'
    ELSE 'Senior'
  END AS age_group
FROM customers;
```

خیلی مهم و کاربردی است.

Conditional Aggregation .50

ترکیب قدرتمند CASE و Aggregate:

```
SELECT
  country,
  COUNT(*) AS total_customers,
  SUM(
    CASE
      WHEN age >= 30 THEN 1
      ELSE 0
    END
  ) AS customers_over_30
FROM customers
GROUP BY country;
```

این Pattern در Data Analysis بسیار پرکاربرد است.

String Functions .51

بسته به DBMS Syntax کمی فرق دارد.

موارد رایج:

```
UPPER(name)
LOWER(name)
LENGTH(name)
TRIM(name)
```

مثلاً:

```
SELECT UPPER(name)
FROM customers;
```

Date Functions .52

یکی از پرکاربردترین بخش‌ها در Analytics.

مثلاً PostgreSQL:

```
SELECT
  EXTRACT(YEAR FROM order_date)
FROM orders;
```

یا:

```
DATE_TRUNC('month', order_date)
```

اما Date Function ها بین:

PostgreSQL •

MySQL •

SQL Server •

Oracle •

اختلاف نسبتاً زیادی دارند.

Subquery .53

Query داخل Query.

مثلاً:

مشتریانی که سنشان بیشتر از میانگین سن کل مشتریان است.

اول میانگین:

```
SELECT AVG(age)
FROM customers;
```

بعد:

```
SELECT *
FROM customers
WHERE age > (
  SELECT AVG(age)
  FROM customers
);
```

قسمت داخلی:

```
SELECT AVG(age)
FROM customers
```

Subquery است.

54. Subquery با IN

مثلاً مشتریانی که حداقل یک سفارش دارند:

```
SELECT *
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

55. EXISTS

راه قدرتمند دیگر:

```
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

یعنی:

مشتری‌هایی که برایشان حداقل یک Order وجود دارد.

برای بعضی Query ها EXISTS از نظر معنایی و گاهی Performance انتخاب بسیار خوبی است.

56. Correlated Subquery

Subquery ای که به Row Query بیرونی وابسته است.

مثلاً:

```
SELECT c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.amount > 1000
);
```

به Subquery:

```
c.customer_id
```

وابسته است.

57. CTE چیست؟

Common Table Expression

با:

```
WITH
```

ساخته می‌شود.

مثلاً:

```
WITH customer_sales AS (
  SELECT
    customer_id,
    SUM(amount) AS total_sales
  FROM orders
  GROUP BY customer_id
)
SELECT *
FROM customer_sales
WHERE total_sales > 1000;
```

CTE باعث می‌شود Query‌های پیچیده خوانا تر شوند.

58. چرا CTE خوب است؟

بدون CTE ممکن است Query به این شکل شود:

```
SELECT *
FROM (
  SELECT ...
  FROM ...
  GROUP BY ...
) x
WHERE ...
```

ولی CTE ساختار منطقی‌تر می‌دهد:

```
WITH sales AS (...),
     active_customers AS (...)
SELECT ...
```

برای Data Analytics بسیار مهم است.

59. Set Operations

SQL می‌تواند Result Set‌ها را ترکیب کند.

مهم‌ترین‌ها:

```
UNION
UNION ALL
```

```
INTERSECT
EXCEPT
```

UNION .60

```
SELECT email
FROM customers

UNION

SELECT email
FROM suppliers;
```

دو Result را ترکیب می‌کند و Duplicate ها را حذف می‌کند.

UNION ALL .61

```
SELECT email
FROM customers

UNION ALL

SELECT email
FROM suppliers;
```

Duplicate ها باقی می‌مانند.

معمولاً سریع‌تر است چون نیازی به Duplicate Elimination ندارد.

62. تفاوت UNION و JOIN

خیلی مهم:

JOIN جدول‌ها را افقی کنار هم می‌گذارد:

```
customer + order
```

ولی UNION Result ها را عمودی زیر هم قرار می‌دهد.

مثلاً:

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers
```

Window Functions .63

اینجا وارد SQL پیشرفته می‌شویم.

یکی از مهم‌ترین مفاهیم SQL Analytics.

مثلاً فرض کن:

person	salary
Ali	5000
Sara	7000
John	6000

می‌خواهیم Rank بدهیم.

```
SELECT
  name,
  salary,
  RANK() OVER (
    ORDER BY salary DESC
  ) AS salary_rank
FROM employees;
```

خروجی:

```
Sara  7000  1
John  6000  2
Ali   5000  3
```

64. فرق GROUP BY و Window Function

بسیار بسیار مهم.

GROUP BY تعداد Rowها را کاهش می‌دهد.

مثلاً:

```
SELECT country, AVG(salary)
FROM employees
GROUP BY country;
```

ممکن است Employee 1000 را تبدیل کند به 10 Country.

ولی Window Function:

Rowها را نگه می‌دارد.

مثلاً:

```
SELECT
  name,
  country,
  salary,
```

```
AVG(salary) OVER (
  PARTITION BY country
) AS country_avg_salary
FROM employees;
```

هر Employee باقی می ماند ولی Average کشور هم کنارش می آید.

PARTITION BY .65

مثلاً Rank در هر Department:

```
SELECT
  name,
  department,
  salary,
  RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS dept_rank
FROM employees;
```

Rank برای هر Department از نو شروع می شود.

ROW_NUMBER / RANK / DENSE_RANK .66

از سؤال های کلاسیک مصاحبه.

فرض کنیم Salary:

```
100
100
90
80
```

ROW_NUMBER

```
1
2
3
4
```

حتی Tie ها شماره متفاوت دارند.

RANK

```
1
1
3
4
```

بعد از Tie شکاف ایجاد می شود.

DENSE_RANK

```
1
1
2
3
```

شکاف ایجاد نمی‌شود.

67. مثال معروف مصاحبه

دومین حقوق بالاتر را پیدا کن.

یک راه:

```
SELECT salary
FROM (
    SELECT
        salary,
        DENSE_RANK() OVER (
            ORDER BY salary DESC
        ) AS rnk
    FROM employees
) x
WHERE rnk = 2;
```

68. LEAD و LAG

برای Time Series عالی‌اند.

مثلاً فروش روز قبل:

```
SELECT
    order_date,
    sales,
    LAG(sales) OVER (
        ORDER BY order_date
    ) AS previous_day_sales
FROM daily_sales;
```

می‌توانیم تغییر را حساب کنیم:

```
SELECT
    order_date,
    sales,
    sales - LAG(sales) OVER (
        ORDER BY order_date
    ) AS change
FROM daily_sales;
```

Running Total .69

```
SELECT
  order_date,
  amount,
  SUM(amount) OVER (
    ORDER BY order_date
  ) AS running_total
FROM orders;
```

مثلاً:

```
100 → 100
200 → 300
50  → 350
```

SQL Command Categories .70

به طور کلاسیک دستورات SQL را این طور دسته بندی می کنند:

دسته	مفهوم	نمونه
DQL	Query Data	SELECT
DML	Modify Data	INSERT, UPDATE, DELETE
DDL	Structure	CREATE, ALTER, DROP
DCL	Permissions	GRANT, REVOKE
TCL	Transactions	COMMIT, ROLLBACK

این طبقه بندی در مصاحبه زیاد پرسیده می شود.

Transaction .71 چیست؟

فرض کن انتقال پول انجام می دهی:

```
Shayan → -100 €
Ali    → +100 €
```

دو عملیات باید با هم انجام شوند.

اگر اولی انجام شود ولی سیستم Crash کند قبل از دومی چه؟

پول گم می شود.

برای همین Transaction داریم.

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

اگر مشکل باشد:

```
ROLLBACK;
```

ACID .72

یکی از اصلی ترین مفاهیم Database.

A — Atomicity

Transaction یا کامل انجام می شود یا اصلاً انجام نمی شود.

```
All or Nothing
```

C — Consistency

Database قبل و بعد Transaction باید قوانین خودش را حفظ کند.

مثلاً Foreign Key نباید خراب شود.

I — Isolation

Transaction های همزمان نباید به شکل نامناسب روی هم اثر بگذارند.

D — Durability

بعد از Commit داده باید پایدار بماند حتی اگر سیستم Crash کند.

پس:

```
ACID
Atomicity
Consistency
Isolation
Durability
```

حتماً حفظش کن.

73. Concurrency Problem ها

وقتی چند User همزمان کار می کنند ممکن است مشکلاتی مانند:

```
Dirty Read
Non-repeatable Read
Phantom Read
Lost Update
```

به وجود بیاید.

Database ها با Isolation Level ها با این موضوع برخورد می کنند.

مثلاً:

```
Read Uncommitted
Read Committed
Repeatable Read
Serializable
```

جزئیات دقیق و رفتار واقعی DBMS ها متفاوت است.

برای مصاحبه مقدماتی معمولاً دانستن مفهوم کافی است.

74. Index چیست؟

یکی از مهم ترین مفاهیم Performance.

فرض کن کتاب 2 میلیون صفحه ای داری و دنبال:

```
Shayan Rokhva
```

هستی.

اگر Index نداشته باشی شاید مجبور باشی همه صفحات را بررسی کنی.

Index تقریباً مانند فهرست کتاب است.

مثلاً:

```
CREATE INDEX idx_customers_email
ON customers(email);
```

بعد Query:

```
SELECT *
FROM customers
WHERE email = 'x@example.com';
```

می تواند بسیار سریع تر شود.

75. چرا برای همه Column ها Index نمی‌گذاریم؟

چون Index مجانی نیست.

مزایا:

```
SELECT سریع تر
Search سریع تر
JOIN سریع تر
```

معایب:

```
Storage بیشتر
INSERT کند تر
UPDATE کند تر
DELETE کند تر
Maintenance
```

پس Index یک **Trade-off** است.

76. چه ستون‌هایی Candidate خوب Index هستند؟

اغلب ستون‌هایی که زیاد در این‌ها استفاده می‌شوند:

```
WHERE
JOIN
ORDER BY
```

یا Foreign Key ها و Search Column ها.

ولی Context مهم است.

مثلاً Index روی Boolean Column با فقط دو مقدار:

```
TRUE / FALSE
```

ممکن است خیلی مفید نباشد، مخصوصاً اگر Selectivity پایین باشد.

77. Composite Index

مثلاً:

```
CREATE INDEX idx_customer_country_age
ON customers(country, age);
```

ترتیب Column ها مهم است.

برای مثال در بسیاری از B-tree implementations، چنین Index می‌تواند برای Query‌هایی که از prefix یعنی country استفاده می‌کنند مفید باشد، ولی لزوماً برای Query صرفاً روی age به همان اندازه مفید نیست. این موضوع به Query Planner و DBMS بستگی دارد.

Query Planner .78

SQL Declarative است.

مثلاً تو می‌نویسی:

```
SELECT *
FROM customers
WHERE email = 'abc@test.com';
```

Database تصمیم می‌گیرد:

```
Table Scan?
Index Scan?
Which Join algorithm?
Which table first?
```

این تصمیم توسط:

```
Query Optimizer
```

گرفته می‌شود.

EXPLAIN .79

برای فهمیدن Query Plan.

مثلاً PostgreSQL:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'abc@test.com';
```

و برای اجرای واقعی و اندازه‌گیری:

```
EXPLAIN ANALYZE
SELECT *
FROM customers
WHERE email = 'abc@test.com';
```

ابزار بسیار مهم Performance Tuning است.

Join Algorithms .80

خود DBMS می تواند Join را با الگوریتم های مختلف اجرا کند:

```
Nested Loop Join
Hash Join
Merge Join
```

تو معمولاً مستقیم Algorithm را نمی نویسی؛ Optimizer تصمیم می گیرد.

ولی برای سطح حرفه ای Database دانستن این ها مهم است.

Normalization .81

یک مفهوم Database Design

فرض کنیم یک جدول بد داریم:

order_id	customer_name	customer_phone	product	product_price
----------	---------------	----------------	---------	---------------

اگر یک مشتری 100 سفارش بدهد، Name و Phone صد بار تکرار می شوند.

این طراحی مشکل دارد.

بهتر:

```
customers
-----
customer_id
name
phone

products
-----
product_id
name
price

orders
-----
order_id
customer_id

order_items
-----
order_id
product_id
quantity
```

این کار تکرار داده را کاهش می دهد.

82. چرا Normalization؟

برای کاهش:

Data Redundancy
Update Anomaly
Insert Anomaly
Delete Anomaly

83. Normal Forms

معروفترین:

1NF
2NF
3NF
BCNF

برای یک مصاحبه Data Science معمولاً فهم 1NF–3NF کافی است.

84. 1NF

به شکل ساده:

هر Cell باید یک مقدار Atomic داشته باشد.

بد:

id	phones
1	0912,0935,021

بتر:

جدول جدا برای Phone ها.

85. 2NF

باید 1NF برقرار باشد و Attribute های non-key وابستگی جزئی به بخشی از Composite Key نداشته باشند.

این بیشتر وقتی مطرح است که Primary Key مرکب داری.

3NF .86

تقریباً:

Non-key Attribute نباید از طریق Non-key Attribute دیگری به Primary Key وابسته باشد.

مثلاً:

```
student_id
department_id
department_name
```

department_name در اصل به department_id وابسته است.

پس بهتر است Department جدول جدا داشته باشد.

Denormalization .87

Normalization همیشه آخر داستان نیست.

گاهی برای Analytics یا Performance عمداً داده را تکرار می‌کنیم.

این می‌شود:

```
Denormalization
```

مثلاً Data Warehouse ها اغلب ساختارهایی مانند:

```
Star Schema
```

دارند.

OLTP vs OLAP .88

خیلی مفید است بدانی.

OLTP

Online Transaction Processing

مثل:

```
Banking
E-commerce transactions
Reservation
```

ویژگی:

- Transaction های زیاد
- کوچک

- سریع
- Insert/Update زیاد

OLAP

Online Analytical Processing

برای:

```
Analytics
BI
Reporting
Data Warehouse
```

Query ها معمولاً:

- بزرگتر
- Aggregate-heavy
- خواندن زیاد

مثلاً:

فروش سه سال اخیر به تفکیک Country, Product, Month.

View .89

یک Query ذخیره شده که مانند Table استفاده می شود.

مثلاً:

```
CREATE VIEW high_value_customers AS
SELECT
    customer_id,
    SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

بعد:

```
SELECT *
FROM high_value_customers;
```

View معمولاً خودش داده مستقل ذخیره نمی کند؛ Definition Query را نگه می دارد، هرچند Materialized View داستان متفاوتی دارد.

Materialized View .90

برخلاف View معمولی، Result را ذخیره می‌کند.

مزیت:

سریع تر Query

عیب:

شود stale ممکن است Data
Needs refresh

PostgreSQL مثلاً Materialized View دارد.

Stored Procedure .91

بعضی DBMS ها اجازه می‌دهند Logic را داخل Database ذخیره کنی.

مثلاً:

Stored Procedure
Stored Function
Trigger

این‌ها بیشتر در Backend/Database Engineering مهم می‌شوند.

برای Data Analyst دانستن Concept کافی است.

Trigger .92

Trigger به صورت خودکار در واکنش به Event اجرا می‌شود.

مثلاً:

AFTER INSERT
BEFORE UPDATE
AFTER DELETE

کاربرد:

• Audit

• Validation

• بعضی Business Logic ها

ولی استفاده بیش از حد از Trigger می‌تواند Logic سیستم را پنهان و Debug را سخت کند.

SQL Injection .93

موضوع امنیتی بسیار مهم.

این کد خطرناک است:

```
query = "SELECT * FROM users WHERE username = '" + username + "'"
```

کاربر ممکن است Input مخرب وارد کند.

مثلاً مشهورترین شکل مفهومی:

```
' OR 1=1 --
```

برای مقابله باید از:

```
Parameterized Queries  
Prepared Statements
```

استفاده شود.

مثلاً نه String Concatenation.

SQL Case Sensitivity .94

Keywordها معمولاً Case-insensitive هستند:

```
SELECT  
select  
Select
```

معمولاً تفاوت ندارند.

ولی Convention رایج:

```
SELECT name  
FROM customers  
WHERE age > 30;
```

یعنی Keyword با uppercase.

Table/column naming behavior و identifier case بسته به DBMS متفاوت است.

Comments .95

یک خط:

```
-- This is a comment  
SELECT * FROM customers;
```

چندخطی:

```
/*  
Multi-line  
comment  
*/
```

Naming Convention .96

بهتر:

```
customer_id  
order_date  
product_name
```

بدتر:

```
CustomerIDValueNew  
X1  
my-column
```

یک Convention انتخاب کن و consistent بمان.

معمولاً:

```
snake_case
```

خیلی رایج است.

97. مهم‌ترین Keyword های SQL

اگر بخواهم فقط مهم‌ترین‌ها را جدا کنم:

```
SELECT  
FROM  
WHERE  
DISTINCT  
ORDER BY  
LIMIT  
  
GROUP BY  
HAVING  
  
INNER JOIN  
LEFT JOIN  
  
INSERT  
UPDATE  
DELETE  
  
CREATE  
ALTER  
DROP  
  
COUNT  
SUM  
AVG  
MIN  
MAX
```

```
CASE
WHEN
THEN
ELSE
END

IN
BETWEEN
LIKE
IS NULL

EXISTS

WITH

UNION
UNION ALL

OVER
PARTITION BY
ROW_NUMBER
RANK
DENSE_RANK
LAG
LEAD

BEGIN
COMMIT
ROLLBACK
```

اگر این‌ها را واقعاً بلد باشی، بخش بسیار بزرگی از SQL کاربردی را پوشش می‌دهی.

98. یک دیتابیس نمونه واقعی

فرض کنیم فروشگاه داریم.

customers

```
customer_id
name
country
email
signup_date
```

products

```
product_id
product_name
category
price
```

orders

```
order_id
customer_id
order_date
status
```

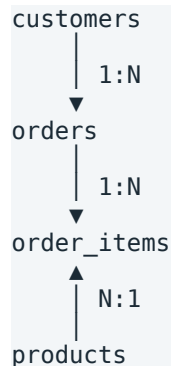
order_items

```

order_id
product_id
quantity
unit_price

```

:Relation



99. مثال جامع اول

می‌خواهیم سفارش‌ها را با Customer ببینیم.

```

SELECT
    o.order_id,
    c.name,
    o.order_date,
    o.status
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id;

```

100. مبلغ هر Order

مبلغ سفارش در order_items است:

```
quantity * unit_price
```

پس:

```

SELECT
    order_id,
    SUM(quantity * unit_price) AS total_amount
FROM order_items
GROUP BY order_id;

```

101. کل خرید هر Customer

حالا سه Table را Join می‌کنیم:

```
SELECT
    c.customer_id,
    c.name,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    c.customer_id,
    c.name;
```

این Query بسیار مهم است.

102. فقط Customer هایی که بیش از 5000 خرید داشته‌اند

```
SELECT
    c.customer_id,
    c.name,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    c.customer_id,
    c.name
HAVING
    SUM(oi.quantity * oi.unit_price) > 5000;
```

103. مرتب‌سازی از بیشترین خرید

```
ORDER BY total_spent DESC;
```

Query نهایی:

```
SELECT
    c.customer_id,
    c.name,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY
    c.customer_id,
    c.name
HAVING
    SUM(oi.quantity * oi.unit_price) > 5000
ORDER BY
    total_spent DESC;
```

اگر این Query را کاملاً بفهمی، بخش قابل توجهی از SQL را گرفته‌ای.

104. پرفروش ترین Product

```
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS units_sold
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY
    p.product_id,
    p.product_name
ORDER BY
    units_sold DESC
LIMIT 10;
```

105. مشتریانی که هیچ سفارشی ندارند

Pattern بسیار معروف مصاحبه.

```
SELECT
    c.customer_id,
    c.name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

منطق:

LEFT JOIN تمام Customer ها را حفظ می کند.

اگر Order نداشته باشند:

```
o.order_id = NULL
```

پس آن ها را انتخاب می کنیم.

راه دیگر:

```
SELECT *
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

106. فروش بر اساس کشور

```
SELECT
    c.country,
    SUM(oi.quantity * oi.unit_price) AS revenue
```

```
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  c.country
ORDER BY
  revenue DESC;
```

این دقیقاً جنس Query های Data Analyst است.

107. فروش ماهانه

در PostgreSQL:

```
SELECT
  DATE_TRUNC('month', o.order_date) AS month,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  DATE_TRUNC('month', o.order_date)
ORDER BY
  month;
```

108. تغییر فروش نسبت به ماه قبل

سطح بالاتر:

```
WITH monthly_sales AS (
  SELECT
    DATE_TRUNC('month', o.order_date) AS month,
    SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  GROUP BY
    DATE_TRUNC('month', o.order_date)
)
SELECT
  month,
  revenue,
  LAG(revenue) OVER (
    ORDER BY month
  ) AS previous_month_revenue,
  revenue - LAG(revenue) OVER (
    ORDER BY month
  ) AS absolute_change
FROM monthly_sales;
```

این دیگر SQL Analytics واقعی است.

109. درصد رشد ماهانه

```
WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', o.order_date) AS month,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM orders o
    JOIN order_items oi
    ON o.order_id = oi.order_id
    GROUP BY
        DATE_TRUNC('month', o.order_date)
),
sales_with_previous AS (
    SELECT
        month,
        revenue,
        LAG(revenue) OVER (
            ORDER BY month
        ) AS previous_revenue
    FROM monthly_sales
)
SELECT
    month,
    revenue,
    previous_revenue,
    100.0 * (revenue - previous_revenue)
    / NULLIF(previous_revenue, 0) AS growth_percent
FROM sales_with_previous;
```

Division by Zero جلوگیری (NULLIF(previous_revenue, 0)) را می‌گیرد.

110. Duplicate Data پیدا کردن

مثلاً Email‌های تکراری:

```
SELECT
    email,
    COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

یکی از مهم‌ترین Query‌های Data Cleaning.

111. چرا COUNT(*) و COUNT(column) متفاوت‌اند؟

فرض کن:

```
age
---
20
NULL
30
COUNT(*)
```

می‌شود:

```
3
```

ولی:

```
COUNT (age)
```

می‌شود:

```
2
```

چون `COUNT(column)` معمولاً `NULL` را نمی‌شمارد.

این سؤال محبوب مصاحبه است.

112. NULL و AVG

اگر:

```
10  
20  
NULL
```

داشته باشیم:

```
AVG(value)
```

معمولاً Average از:

```
10, 20
```

گرفته می‌شود:

```
15
```

نه:

```
10
```

یعنی `NULL` معمولاً `ignore` می‌شود.

113. WHERE و LEFT JOIN — دام بسیار مهم

این Query را ببین:

```
SELECT *  
FROM customers c  
LEFT JOIN orders o  
  ON c.customer_id = o.customer_id  
WHERE o.status = 'paid';
```

مشکل چیست؟

Rowهایی که Order ندارند:

```
o.status = NULL
```

و توسط WHERE حذف می‌شوند.

بنابراین LEFT JOIN ممکن است عملاً شبیه INNER JOIN شود.

اگر می‌خواهیم فقط Paid Order را Join کنیم ولی Customer‌ها حفظ شوند:

```
SELECT *
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
 AND o.status = 'paid';
```

این نکته واقعاً مهم است.

114. SQL Query‌های خطرناک

این‌ها را هیچ‌وقت بدون فکر اجرا نکن:

```
DELETE FROM customers;
```

یا:

```
UPDATE customers
SET status = 'inactive';
```

وقتی قصدت یک Row بوده.

عادت حرفه‌ای:

اول:

```
SELECT *
FROM customers
WHERE ...;
```

مطمئن شو Row‌های درست را گرفته‌ای.

بعد همان WHERE را در UPDATE/DELETE استفاده کن.

115. Transaction برای تغییرات مهم

مثلاً:

```
BEGIN;

UPDATE customers
SET status = 'inactive'
WHERE last_login < '2024-01-01';

-- inspect

ROLLBACK;
```

اگر مطمئن شدی:

```
COMMIT;
```

این روش برای کار حساس بسیار مهم است.

116. SQL Performance — چند قانون کاربردی

فرض کن جدول 100 میلیون Row دارد.

Query ضعیف:

```
SELECT *  
FROM orders;
```

اگر واقعاً همه داده را نمی‌خواهی، وحشتناک است.

بهتر:

```
SELECT  
    order_id,  
    customer_id,  
    amount  
FROM orders  
WHERE order_date >= '2026-01-01';
```

همچنین Index مناسب:

```
CREATE INDEX idx_orders_date  
ON orders(order_date);
```

ممکن است کمک بزرگی کند.

117. SARGability

مفهوم کمی حرفه‌ای‌تر.

فرض کن Index روی `order_date` داری.

این Query:

```
WHERE order_date >= '2026-01-01'
```

معمولاً Index-friendly است.

اما:

```
WHERE YEAR(order_date) = 2026
```

بسته به DBMS ممکن است استفاده از Index را سخت‌تر کند.

اغلب بهتر:

```
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
```

به چنین Predicate هایی اصطلاحاً SARGable می‌گویند.

118. SELECT * چرا همیشه خوب نیست؟

مثلاً Table:

100 Columns

ولی فقط:

```
name
email
```

می‌خواهی.

این:

```
SELECT *
```

ممکن است:

- I/O بیشتر
- Network بیشتر
- Memory بیشتر
- Coupling بیشتر
- Query Plan متفاوت

ایجاد کند.

119. N+1 Problem

موضوع مهم Backend.

فرض کن:

```
1 query → users
100 queries → orders of each user
```

یعنی 101 Query.

گاهی با یک JOIN می‌شود:

```
SELECT ...
FROM users
JOIN orders ...
```

این مشکل در ORM ها زیاد دیده می شود.

120. SQL و Python چه ارتباطی دارند؟

در Data Science خیلی رایج:

```
Database
  ↓ SQL
Extract Data
  ↓
Python / Pandas
  ↓
Analysis / ML
```

مثلاً:

```
pd.read_sql(query, connection)
```

بنابراین SQL برای Data Scientist تقریباً Skill ضروری است.

121. SQL vs Pandas

مثلاً SQL:

```
SELECT
    country,
    AVG(age)
FROM customers
GROUP BY country;
```

Pandas:

```
df.groupby("country")["age"].mean()
```

مفهومها بسیار شبیه اند.

122. Mapping مفاهیم SQL و Pandas

SQL	Pandas
SELECT	column selection
WHERE	boolean filtering
GROUP BY	groupby
JOIN	merge

ORDER BY	sort_values
COUNT	count
AVG	mean
DISTINCT	unique/drop_duplicates

اگر Pandas را بلدی، SQL بسیار سریع جا می افتد.

SQL vs NoSQL .123

SQL Database ها:

PostgreSQL
MySQL
SQL Server
Oracle

معمولاً relational و schema-oriented هستند.

NoSQL:

MongoDB
Cassandra
Redis
DynamoDB

مدل های متفاوت دارند:

Document
Key-value
Wide-column
Graph

نه اینکه یکی مطلقاً بهتر باشد.

مسئله و workload تعیین می کند.

MySQL یا PostgreSQL .124

برای یادگیری جدی SQL، من معمولاً PostgreSQL را انتخاب بسیار خوبی می دانم.

چون:

- بسیار قدرتمند است.
- Open-source است.

- استاندارد SQL را خوب پشتیبانی می‌کند.
 - Window Function عالی
 - CTE
 - JSON
 - Advanced Data Types
 - Extensions
 - GIS با PostGIS
- MySQL نیز بسیار محبوب است، مخصوصاً در Web Development.
- SQL Server در Enterprise و Microsoft Ecosystem بسیار مهم است.

125. برای تمرین چه محیطی؟

می‌توانی:

PostgreSQL + pgAdmin

یا:

MySQL + MySQL Workbench

یا برای ساده‌ترین شروع:

SQLite

استفاده کنی.

حتی سایت‌هایی مثل LeetCode Database, HackerRank SQL, SQLBolt و DataLemur برای تمرین Query عالی‌اند.

126. یک Query را چطور ذهنی حل کنیم؟

وقتی سؤال می‌دهد:

5 مشتری با بیشترین خرید در سال 2026 را پیدا کن.

نباید فوراً SQL بنویسی.

ابتدا Logical Steps:

1. Table؟ کدام

customers
orders
order_items

2. Relation؟

JOIN

3. فقط سال 2026 ؟

WHERE

4. Customer خرید هر ؟

SUM

5. گروه بندی ؟

GROUP BY customer

6. بیشترین ؟

ORDER BY DESC

7. فقط 5 ؟

LIMIT 5

سپس SQL:

```
SELECT
    c.customer_id,
    c.name,
    SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_items oi
    ON o.order_id = oi.order_id
WHERE
    o.order_date >= '2026-01-01'
    AND o.order_date < '2027-01-01'
GROUP BY
    c.customer_id,
    c.name
ORDER BY
    total_spent DESC
LIMIT 5;
```

این روش حل مسئله را یاد بگیر؛ حفظ Query فایده کمی دارد.

127. مهم‌ترین سؤال‌های مصاحبه SQL

اگر برای مصاحبه آماده می‌شوی، باید بتوانی بدون مکث توضیح بدهی:

Primary Key چیست؟

Row Identifier یکتای

Foreign Key چیست؟

ستونی که Referential Relationship با جدول دیگر ایجاد می‌کند.

WHERE vs HAVING ؟

WHERE قبل از Aggregation؛ HAVING بعد از Grouping.

INNER vs LEFT JOIN ؟

INNER فقط Match؛ LEFT همه LEFT + Matching Right.

§ UNION vs UNION ALL

UNION Duplicate حذف می‌کند، UNION ALL نه.

§ DELETE vs DROP vs TRUNCATE

Rows vs همه Rows سریع‌تر/ساختاری متفاوت vs Object.

? COUNT(*) vs COUNT(column)

دومی NULLها را حساب نمی‌کند.

§ NULL چیست؟

Unknown/Missing، نه صفر و نه empty string.

§ Primary Key vs UNIQUE

هر دو uniqueness می‌دهند، ولی Primary Key شناسه اصلی Table و non-null است؛ جزئیات UNIQUE/NULL بسته به DBMS متفاوت است.

§ Clustered vs Non-clustered Index

پیاده‌سازی و اصطلاحات DBMS-dependent است؛ در SQL Server مثلاً Clustered Index ترتیب فیزیکی/ساختار داده جدول را تعیین می‌کند و فقط یکی می‌تواند وجود داشته باشد، درحالی‌که Nonclustered ساختار جداگانه است.

§ GROUP BY vs Window Function

GROUP BY Rowها را collapse می‌کند؛ Window Function Rowها را حفظ می‌کند.

128. SQL اشتباه مبتدی‌ها

چند مورد را جدی بگیر:

1. مناسب ON بدون JOIN
2. Forgetting WHERE in UPDATE/DELETE
3. WHERE و HAVING اشتباه گرفتن
4. = استفاده از NULL
5. duplicate rows after JOIN نفهمیدن
6. SELECT * everywhere
7. Bug برای پنهان کردن DISTINCT استفاده بی‌فکر از
8. زیاد Index
9. نداشتن Index
10. data types عدم توجه به
11. برای پول FLOAT استفاده از
12. روابط cardinality نفهمیدن
13. transaction نادیده گرفتن
14. concatenating SQL strings
15. execution plan نادیده گرفتن

مورد شماره 7 خیلی مهم است.

اگر JOIN اشتباه باعث duplicate شده، این:

```
SELECT DISTINCT ...
```

ممکن است فقط Bug را مخفی کند.

اول بفهم چرا Duplicate ایجاد شده.

129. Cardinality چیست؟

در Database Design روابط می توانند باشند:

One-to-One

```
Person → Passport
```

One-to-Many

```
Customer → Orders
```

یک Customer، چند Order.

Many-to-Many

```
Students ↔ Courses
```

یک Student چند Course.

یک Course چند Student.

برای Many-to-Many معمولاً Junction Table داریم:

```
student_courses
-----
student_id
course_id
```

130. Composite Primary Key

مثلاً:

```
student_courses
```

می تواند Primary Key مرکب داشته باشد:

```
PRIMARY KEY (student_id, course_id)
```

یعنی Pair باید Unique باشد.

Surrogate Key vs Natural Key .131

:Natural Key

مثلاً:

```
national_id
email
```

چیزی که در Domain معنی دارد.

:Surrogate Key

```
customer_id = 12345
```

صرفاً Identifier مصنوعی.

در بسیاری از سیستم‌ها Surrogate Key ترجیح داده می‌شود چون پایدارتر و ساده‌تر است.

Schema مثال خوب .132

مثلاً:

```
CREATE TABLE customers (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  country VARCHAR(100),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

.PostgreSQL-like syntax

Identity / Auto Increment .133

لازم نیست ID را دستی تعیین کنی.

:MySQL

```
AUTO_INCREMENT
```

:جدید PostgreSQL

```
GENERATED ... AS IDENTITY
```

:SQL Server

```
IDENTITY
```

Dialectها فرق دارند.

134. یک سؤال مهم: SQL زبان برنامه‌نویسی است؟

SQL یک زبان کامپیوتری تخصصی و Declarative است، اما معمولاً آن را مثل Python/C++ یک General-purpose imperative programming language در نظر نمی‌گیرند.

با Extension‌هایی مثل:

```
PL/pgSQL  
T-SQL  
PL/SQL
```

می‌توان Logic برنامه‌نویسی بیشتری داشت.

135. SQL واقعاً چقدر مهم است؟

اگر مسیری از این‌ها باشد:

```
Data Analyst  
Data Scientist  
Data Engineer  
Backend Developer  
ML Engineer  
Business Intelligence  
Database Engineer
```

SQL بسیار مهم است.

برای Data Analyst حتی می‌توان گفت جزو Skill‌های اصلی است.

برای Data Scientist نیز در شرکت‌های واقعی، Data Extraction با SQL یک مهارت بسیار رایج است.

136. سطح‌بندی واقعی SQL

من SQL را تقریباً این‌طور سطح‌بندی می‌کنم:

Level 1 — Basic

```
SELECT  
WHERE  
ORDER BY  
LIMIT  
DISTINCT
```

Level 2 — Intermediate

```
GROUP BY  
HAVING  
Aggregate  
JOIN  
CASE  
NULL
```

Level 3 — Strong

- Subqueries
- CTE
- UNION
- EXISTS
- Window Functions
- Complex joins
- Date analytics

Level 4 — Advanced

- Query plans
- Indexes
- Transactions
- Isolation
- Normalization
- Performance tuning
- Recursive CTE
- Partitioning
- Materialized Views
- Database internals

Level 5 — Database Specialist

- Storage engines
- B-tree internals
- MVCC
- Locks
- WAL
- Recovery
- Replication
- Sharding
- Distributed SQL
- Query Optimizer internals
- Cost models

برای مصاحبه Data Science معمولاً **Level 3** خوب + مقداری **Level 4** کافی است.

137. چیزی که باید واقعاً حفظ باشی

اگر از تمام این آموزش فقط یک Skeleton یادت بماند، این باشد:

```
SELECT
  ...
FROM
  table1
JOIN
  table2
  ON ...
WHERE
  ...
GROUP BY
  ...
HAVING
  ...
ORDER BY
  ...
LIMIT ...;
```

و ترتیب منطقی:

```

FROM / JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
↓
LIMIT

```

این ستون فقرات SQL است.

Cheat Sheet .138 بسیار مهم

```

-- Read
SELECT column1, column2
FROM table
WHERE condition
ORDER BY column DESC
LIMIT 10;

-- Unique values
SELECT DISTINCT country
FROM customers;

-- Aggregate
SELECT
    country,
    COUNT(*) AS n,
    AVG(age) AS avg_age
FROM customers
GROUP BY country
HAVING COUNT(*) > 10;

-- Join
SELECT
    c.name,
    o.order_id
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id;

-- Left Join
SELECT
    c.name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;

-- Insert
INSERT INTO customers(name, age)
VALUES ('Ali', 30);

```

```
-- Update
UPDATE customers
SET age = 31
WHERE customer_id = 1;

-- Delete
DELETE FROM customers
WHERE customer_id = 1;

-- Null
WHERE phone IS NULL

-- Case
CASE
  WHEN score >= 90 THEN 'A'
  WHEN score >= 80 THEN 'B'
  ELSE 'C'
END

-- CTE
WITH data AS (
  SELECT ...
)
SELECT *
FROM data;

-- Window
SELECT
  name,
  salary,
  RANK() OVER (
    ORDER BY salary DESC
  ) AS rank
FROM employees;
```

139. پنج Query که اگر بتوانی حل کنی، SQLت بد نیست

فرض کن Tables:

```
customers
orders
order_items
products
```

باید بتوانی حل کنی:

۱. ده مشتری با بیشترین خرید

```
SELECT
  c.customer_id,
  c.name,
  SUM(oi.quantity * oi.unit_price) AS total
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
```

```
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  c.customer_id,
  c.name
ORDER BY total DESC
LIMIT 10;
```

۲. مشتری بدون سفارش

```
SELECT c.*
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

۳. تعداد سفارش هر مشتری

```
SELECT
  c.customer_id,
  c.name,
  COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.name;
```

نکته:

```
COUNT(o.order_id)
```

بهتر از COUNT(*) است اگر می‌خواهی Customer بدون Order مقدار صفر بگیرد.

۴. دومین حقوق بالاتر

با DENSE_RANK

۵. فروش ماهانه و رشد نسبت به ماه قبل

با:

```
GROUP BY + CTE + LAG
```

اگر این‌ها را بدون کمک حل کنی، دیگر Beginner نیستی.

140. نکته بسیار مهم در JOIN و COUNT

مثلاً:

```
SELECT
  c.name,
  COUNT(*)
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.name;
```

برای Customer بدون COUNT، Order (*) ممکن است 1 بدهد چون Row مربوط به Customer بعد از LEFT JOIN همچنان وجود دارد.

برای تعداد Order:

```
COUNT(o.order_id)
```

استفاده کن.

چون NULL شمرده نمی‌شود.

این نکته‌ی مصاحبه‌ای خوبی است.

Recursive CTE .141

سطح پیشرفته‌تر.

برای ساختار Tree:

```
CEO
├── Manager A
│   ├── Employee
│   └── Employee
└── Manager B
```

می‌توان از Recursive CTE استفاده کرد.

مثلاً PostgreSQL:

```
WITH RECURSIVE employee_tree AS (
    SELECT
        employee_id,
        name,
        manager_id,
        1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.employee_id,
        e.name,
        e.manager_id,
        et.level + 1
    FROM employees e
    JOIN employee_tree et
        ON e.manager_id = et.employee_id
)
SELECT *
FROM employee_tree;
```

برای شروع لازم نیست مسلط باشی ولی خوب است بدانی وجود دارد.

Partitioning .142

اگر Table میلیاردها Row داشته باشد، ممکن است آن را Partition کنیم.

مثلاً Orders بر اساس Year:

```
orders_2024
orders_2025
orders_2026
```

ولی از دید Logical ممکن است یک Table باشد.

مزایا در بعضی Workload ها:

- Query pruning
- Maintenance
- Data lifecycle

اما Partitioning خودکار Performance Magic نیست؛ طراحی باید درست باشد.

View امنیتی .143

مثلاً Table کامل:

```
employee
salary
national_id
email
```

ولی Analyst نباید Salary/National ID ببیند.

می‌توان View داد:

```
CREATE VIEW employee_public AS
SELECT
    employee_id,
    name,
    department
FROM employees;
```

و Permission فقط روی View.

GRANT / REVOKE .144

Permission

```
GRANT SELECT
ON customers
TO analyst;
```

برداشتن:

```
REVOKE SELECT
ON customers
FROM analyst;
```

اصل امنیتی مهم:

Principle of Least Privilege

یعنی هر User فقط Permission لازم را داشته باشد.

145. آیا ترتیب Conditions در WHERE مهم است؟

از نظر منطق:

```
WHERE age > 30
AND country = 'Germany'
```

با:

```
WHERE country = 'Germany'
AND age > 30
```

یک نتیجه دارد.

Optimizer معمولاً ترتیب Execution را خودش تعیین می‌کند.

بنابراین فکر نکن شرط اول الزاماً اول اجرا می‌شود.

146. آیا ترتیب JOINها مهم است؟

در Query Result باید Semantics را درست بنویسی، ولی Optimizer در بسیاری موارد می‌تواند Join Order فیزیکی را تغییر دهد.

مخصوصاً برای INNER JOIN.

Outer Joinها محدودیت‌های معنایی بیشتری دارند.

این تفاوت بین:

```
Logical query
```

و:

```
Physical execution plan
```

خیلی مهم است.

147. Database Engine پشت صحنه چه می‌کند؟

وقتی Query می‌فرستی:

```
SELECT *
FROM customers
WHERE age > 30;
```

تقریباً:

```
SQL Text
↓
Parser
↓
Semantic Analysis
↓
Query Rewrite
↓
Optimizer
↓
Execution Plan
↓
Execution Engine
↓
Storage Engine
↓
Result
```

این مدل ذهنی برای سطح بالاتر عالی است.

148. SQL Declarative بودن دقیقاً یعنی چه؟

مثلاً تو نمی‌گویی:

```
Row 1 کن نگاه کن.
Row 2 کن نگاه کن.
کن append بود age > 30 اگر
...
```

بلکه:

```
SELECT *
FROM customers
WHERE age > 30;
```

یعنی چه نتیجه‌ای می‌خواهی.

DBMS تعیین می‌کند چگونه به آن برسد.

149. بهترین روش یادگیری SQL

اگر فقط این متن را بخوانی، SQL یاد نمی‌گیری.

SQL مثل رانندگی است.

باید Query بنویسی.

پیشنهاد من:

مطالعه 20%
80% Query Writing

یعنی هر مفهوم را بلافاصله اجرا کن.

مثلاً بعد از JOIN، حداقل 15-20 JOIN بنویس.

150. مسیر پیشنهادی برای فول شدن

اگر هدف ت مصاحبه است:

مرحله اول

کاملاً مسلط شو به:

SELECT
WHERE
ORDER BY
GROUP BY
HAVING
JOIN
CASE
NULL
Aggregate

مرحله دوم

بعد:

Subquery
CTE
EXISTS
UNION
Window Functions

مرحله سوم

بعد:

Index
Execution Plan
Normalization
Transaction
ACID
Isolation

مرحله چهارم

حداقل:

50-100 SQL problems

حل کن.

اینجاست که Skill واقعی ساخته می‌شود.

جمع‌بندی نهایی

اگر بخواهم تمام **SQL** را در یک تصویر ذهنی خلاصه کنم:

Database مجموعه‌ای از Table‌های مرتبط است.

مثلاً:

Customers
Products
Orders
Payments

SQL زبانی است که با آن به Database می‌گوییم:

را بده Data	→ SELECT
فیلتر کن	→ WHERE
مرتب کن	→ ORDER BY
گروه‌بندی کن	→ GROUP BY
گروه را فیلتر کن	→ HAVING
جدول‌ها را وصل کن	→ JOIN
اضافه کن	→ INSERT
تغییر بده	→ UPDATE
پاک کن	→ DELETE
ساختار بساز	→ CREATE
ساختار تغییر بده	→ ALTER
ساختار حذف کن	→ DROP

سپس SQL پیشرفته اضافه می‌کند:

Subquery
CTE
Window Functions
Transactions
Indexes
Views
Constraints
Query Optimization

و SQL حرفه‌ای‌تر وارد این دنیا می‌شود:

ACID
Isolation
MVCC
Locking
B-trees
Execution Plans
Partitioning
Replication
Database Internals

مرز واقعی **Beginner** و کسی که SQL بلد است، **SELECT** و **WHERE** نیست. مرز اصلی از جایی شروع می‌شود که بتوانی روابط بین جدول‌ها را درست مدل کنی، **JOIN** را بفهمی، **Aggregate**‌ها را بدون خطا بنویسی، **Window Function** به کار ببری و علت کند بودن یا غلط بودن یک **Query** را تشخیص بدهی.

برای مصاحبه‌ی Data Science / ML / Data Analysis، اگر مطالب بالا را واقعاً بفهمی و بتوانی Query‌های بخش‌های ۹۹ تا ۱۰۹ را بدون نگاه کردن بنویسی، پایه‌ی SQL تو کاملاً قابل دفاع است.